

Environmental Statistics

Lecture 12: CART, Random Forest, Bagging and Boosting

Ikedichi Edward Azuh

Carsten F. Dormann

1. Overview

In environmental and ecological statistics, we are often interested in prediction rather than only inference. Many environmental relationships are nonlinear and involve complex interactions between predictors, such as the relationship between climate variables and species distributions, rainfall patterns and flooding, or environmental gradients and ecosystem responses.

Flexible regression models can capture these complex patterns, but increased flexibility also increases the risk of overfitting. A central challenge in predictive modelling is therefore:

How can we build flexible models that generalize well to new data?

Classification and Regression Trees (CART) provide a flexible approach by dividing the predictor space into regions using recursive binary splits. However, individual trees are often unstable and have high variance: small changes in the data can result in substantially different predictions.

This motivates **ensemble methods**, where multiple trees are combined to create stronger predictive models.

Two important ensemble approaches are:

- **Bagging (bootstrap aggregation)**, where many models are fitted to bootstrap samples and their predictions are averaged to reduce variance.
- **Boosting**, where models are fitted sequentially, with each new model aiming to improve the remaining prediction errors of the current ensemble.

Random Forests extend bagging by introducing additional randomness in the selection of predictors at each tree split, while modern boosting methods such as XGBoost provide powerful prediction approaches for structured environmental datasets.

Although these methods often provide excellent predictive performance, environmental modelling also requires careful evaluation of model reliability. Important considerations include:

- predictive accuracy and generalization to new data;
- appropriate validation strategies;
- calibration of predicted probabilities for classification problems.

In this lecture, we introduce CART and tree-based ensemble methods, focusing on their construction, the principles behind their improvement, and their application to environmental prediction.

In this lecture we cover:

- Classification and Regression Trees (CART) and recursive splitting;
- Tree complexity, pruning, and cross-validation;
- Limitations of individual CART models;
- Bagging and bootstrap aggregation;
- Random Forests and random predictor selection;

- Weighted CART and its relationship to boosting;
- Boosting and Extreme Gradient Boosting (XGBoost);
- Calibration limitations of tree-based prediction models.

2. Classification and Regression Trees (CART)

2.1 Basic idea

Classification and Regression Trees (CART) are non-parametric models that predict a response variable by repeatedly splitting the predictor space.

Unlike linear regression:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

CART does not assume a predefined mathematical relationship. Instead, it divides the predictor space into regions:

$$R_1, R_2, \dots, R_M$$

and predicts the mean response within each region.

The prediction is therefore:

$$\hat{y}(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

where:

- c_m is the mean response in region m ;
- I is an indicator function.

In practice, CART is grown by recursively selecting splits that create the most homogeneous groups of observations. The criteria used for selecting these splits are discussed in Section 2.3.

2.2 Example dataset

Throughout this lecture, we illustrate tree-based machine-learning methods using the **Pima Indians Diabetes Database**.¹

The objective is to develop a classification model that predicts whether a patient has diabetes based on a set of clinical measurements.

The response variable is

$$Y = \begin{cases} 1, & \text{patient has diabetes,} \\ 0, & \text{patient does not have diabetes.} \end{cases}$$

The dataset contains observations for **768 adult female patients** of **Pima Indian heritage**, all aged **21 years or older**.

The predictor variables include several routinely measured clinical characteristics:

- Number of pregnancies,
- Plasma glucose concentration,
- Diastolic blood pressure,
- Triceps skin-fold thickness,

¹The dataset was originally collected by the National Institute of Diabetes and Digestive and Kidney Diseases and first described by Smith et al. (1988), *Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus*, Proceedings of the Annual Symposium on Computer Application in Medical Care, pp. 261–265. The dataset is also available from Kaggle for educational use.

- Serum insulin concentration,
- Body mass index (BMI),
- Diabetes pedigree function, and
- Age.

This dataset is widely used as a benchmark for evaluating classification algorithms because it contains a mixture of continuous predictors, nonlinear relationships, and moderate class imbalance, making it well suited for methods such as CART, Random Forests, and boosting.

```
diabetes <- read.csv("diabetes.csv")
diabetes$Outcome <- factor(diabetes$Outcome, levels = c(0,1))

head(diabetes)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
## 1	6	148	72	35	0	33.6
## 2	1	85	66	29	0	26.6
## 3	8	183	64	0	0	23.3
## 4	1	89	66	23	94	28.1
## 5	0	137	40	35	168	43.1
## 6	5	116	74	0	0	25.6

	DiabetesPedigreeFunction	Age	Outcome
## 1	0.627	50	1
## 2	0.351	31	0
## 3	0.672	32	1
## 4	0.167	21	0
## 5	2.288	33	1
## 6	0.201	30	0

```
str(diabetes)
```

```
## 'data.frame': 768 obs. of 9 variables:
## $ Pregnancies : int 6 1 8 1 0 5 3 10 2 8 ...
## $ Glucose : int 148 85 183 89 137 116 78 115 197 125 ...
## $ BloodPressure : int 72 66 64 66 40 74 50 0 70 96 ...
## $ SkinThickness : int 35 29 0 23 35 0 32 0 45 0 ...
## $ Insulin : int 0 0 0 94 168 0 88 0 543 0 ...
## $ BMI : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ DiabetesPedigreeFunction: num 0.627 0.351 0.672 0.167 2.288 ...
## $ Age : int 50 31 32 21 33 30 26 29 53 54 ...
## $ Outcome : Factor w/ 2 levels "0","1": 2 1 2 1 2 1 2 1 2 2 ...
```

```
# Split into training and testing data
set.seed(123)

train_index <- sample(1:nrow(diabetes), size = 0.7*nrow(diabetes))
train_data <- diabetes[train_index, ]
test_data <- diabetes[-train_index, ]
```

Fit and plot fully grown CART

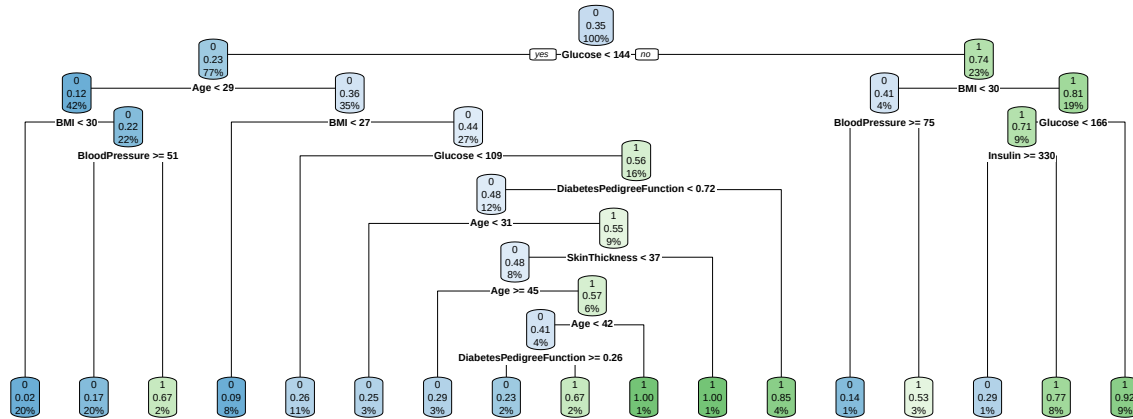
```
library(rpart)
library(rpart.plot)

# Fit a fully grown CART
cart_full <- rpart(
```

```

Outcome ~ .,
data = train_data,
method = "class",
control = rpart.control(
  cp = 0 # to allow the tree to grow to its maximum size
)
# Plot tree
rpart.plot(cart_full, tweak = 1.7)

```



Predict on testdata and evaluate the model accuracy.

```

# Prediction on test data
pred_cart_full <- predict(cart_full, newdata = test_data, type = "class")

library(caret)

confusionMatrix(pred_cart_full, as.factor(test_data$Outcome), positive = "1")

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 127  41
##           1  23  40
##
##           Accuracy : 0.7229
##           95% CI : (0.6604, 0.7796)
##           No Information Rate : 0.6494
##           P-Value [Acc > NIR] : 0.01047
##
##           Kappa : 0.3588
##
##           Mcnemar's Test P-Value : 0.03359
##
##           Sensitivity : 0.4938
##           Specificity : 0.8467
##           Pos Pred Value : 0.6349
##           Neg Pred Value : 0.7560

```

```
##           Prevalence : 0.3506
##       Detection Rate : 0.1732
## Detection Prevalence : 0.2727
##       Balanced Accuracy : 0.6702
##
##       'Positive' Class : 1
##
# Accuracy
accuracy_cart_full <- mean(pred_cart_full == test_data$Outcome)
#accuracy_cart_full
```

2.3 Node impurity and splitting

A CART model is constructed by recursively splitting the predictor space into smaller regions.

Each split divides a parent node into two daughter nodes. The objective is to create daughter nodes that are more homogeneous with respect to the response variable. In other words, CART seeks splits that produce groups containing observations that predominantly belong to a single class.

For classification trees, several measures of node impurity can be used, including entropy, misclassification error, and Gini impurity. CART commonly uses the **Gini impurity** as the splitting criterion.

The Gini impurity is defined as:

$$G = 1 - \sum_{k=1}^K p_k^2,$$

where p_k denotes the proportion of observations belonging to class k .

For the diabetes example, the response variable is binary:

$$Y \in \{0, 1\},$$

therefore the Gini impurity becomes:

$$G = 1 - p_0^2 - p_1^2,$$

where p_0 and p_1 represent the proportions of observations belonging to the non-diabetes and diabetes classes, respectively.

A node containing observations from only one class has $G = 0$, indicating perfect homogeneity (zero impurity). In contrast, a node containing similar proportions of both classes has a larger Gini impurity and is therefore less homogeneous.

At each node, CART evaluates every predictor and every possible split point. For each candidate split, the weighted impurity of the two daughter nodes is computed as:

$$G_{\text{split}} = \frac{n_L}{n} G(\text{left}) + \frac{n_R}{n} G(\text{right}),$$

where:

- n is the number of observations in the parent node;
- n_L and n_R are the numbers of observations in the left and right daughter nodes;
- $G(\text{left})$ and $G(\text{right})$ are the Gini impurities of the two daughter nodes.

The selected split is the one that minimizes the weighted impurity of the daughter nodes. Equivalently, this is the split that produces the greatest increase in node homogeneity.

For regression trees, the response variable is continuous rather than categorical. Therefore, class impurity measures such as the Gini index are not appropriate. Instead, CART evaluates splits according to the prediction error within each daughter node.

The prediction for a terminal node is the mean response of the observations contained in that node:

$$\hat{y}_m = \bar{y}_m.$$

The error within a node is measured using the mean squared error:

$$MSE_m = \frac{1}{n_m} \sum_{i \in R_m} (y_i - \bar{y}_m)^2,$$

where:

- R_m is the set of observations belonging to node m ;
- n_m is the number of observations in node m ;
- y_i is the observed response value for observation i ;
- $\bar{y}_m$ is the mean response of node m and represents the predicted value for observations falling within that node..

For each candidate split, CART calculates the weighted prediction error of the two daughter nodes:

$$MSE_{\text{split}} = \frac{n_L}{n} MSE(\text{left}) + \frac{n_R}{n} MSE(\text{right}).$$

The selected split is the one that minimizes this weighted error. Equivalently, CART chooses the predictor and threshold that produce daughter nodes with the smallest within-node variation.

Therefore, the splitting principle is the same for both types of trees:

Evaluate possible splits $\rightarrow$ Measure node quality $\rightarrow$ Choose the best split.

The only difference is the criterion used to evaluate node quality: classification trees minimize class impurity, while regression trees minimize prediction error.

This process is repeated recursively until the stopping criteria are reached, resulting in a tree that partitions the predictor space into increasingly homogeneous regions.

2.4 CART: step-like regression (for single predictor)

Although this lecture uses a classification example, CART is also widely used for regression problems in environmental statistics.

For regression trees, each terminal node contains a constant prediction:

$$\hat{y}(x) = c_m,$$

where c_m is the mean response of observations within terminal node m .

Therefore, for a single continuous predictor, CART produces a step-like relationship rather than a smooth function.

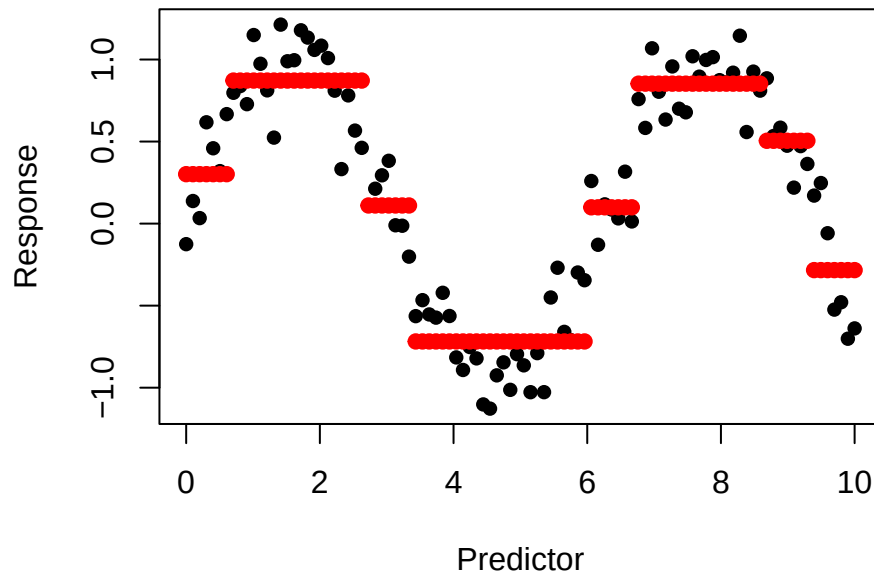
```
library(rpart)

set.seed(1)

x <- seq(0,10,length=100)
y <- sin(x)+rnorm(100,0,0.2)

model <- rpart(y~x)
```

```
plot(x, y, pch=16, xlab="Predictor", ylab="Response")
points(x, predict(model), col="red", pch=19)
```



The number of steps depends on the number of splits in the tree. Increasing the number of splits creates smaller regions and allows the tree to follow the training data more closely.

However, very complex trees may capture random variation rather than the underlying signal. This represents the classical bias–variance trade-off:

simple tree $\rightarrow$ higher bias, lower variance

whereas:

complex tree $\rightarrow$ lower bias, higher variance

This motivates controlling tree complexity through pruning.

2.5 Tree Pruning

The fully grown CART fitted above is intentionally grown with minimal restriction, allowing recursive splitting until the algorithm reaches its stopping criteria. Such a tree usually provides an excellent fit to the training data because it can create many small terminal nodes.

However, a highly complex tree may adapt not only to the underlying structure of the data but also to random variation specific to the training sample. This phenomenon is known as **overfitting**.

The objective of tree pruning is therefore not necessarily to increase training accuracy, but to find a simpler tree that maintains good predictive performance on new observations.

Suppose that T denotes a regression tree consisting of a set of terminal nodes (leaves). The training error

of the tree can be written as:

$$R(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \bar{y}_m)^2,$$

where:

- R_m denotes the observations contained in terminal node m ;
- $\bar{y}_m$ is the mean response within terminal node m ;
- $|T|$ is the number of terminal nodes.

Although the diabetes example is a classification problem, the same cost-complexity principle applies to classification trees. In that case, the error term is based on classification loss measures, such as misclassification error or impurity measures.

Minimizing $R(T)$ alone favours increasingly complex trees because every additional split can reduce the training error. To balance goodness of fit against model complexity, CART uses **cost-complexity pruning**:

$$R_\alpha(T) = R(T) + \alpha|T|,$$

where:

- $R(T)$ measures the lack of fit;
- $|T|$ measures tree complexity;
- $\alpha \geq 0$ is the complexity parameter.

The first term measures the model's lack of fit, whereas the second term penalizes complexity. Increasing α places a stronger penalty on complexity and therefore favours smaller trees.

The complexity parameter table and its corresponding plot summarize the predictive performance of candidate subtrees obtained during cost-complexity pruning:

```
plotcp(cart_full)
```

```
# Complexity parameter table
```

```
printcp(cart_full)
```

```
##
## Classification tree:
## rpart(formula = Outcome ~ ., data = train_data, method = "class",
##       control = rpart.control(cp = 0))
##
## Variables actually used in tree construction:
## [1] Age                      BloodPressure          BMI
## [4] DiabetesPedigreeFunction Glucose                 Insulin
## [7] SkinThickness
##
## Root node error: 187/537 = 0.34823
##
## n= 537
##
##      CP nsplit rel error  xerror   xstd
## 1 0.3155080      0  1.00000 1.00000 0.059037
## 2 0.0213904      1  0.68449 0.68449 0.052800
## 3 0.0196078      2  0.66310 0.73262 0.054021
## 4 0.0178253      7  0.56150 0.73262 0.054021
## 5 0.0160428     10  0.50802 0.70588 0.053356
## 6 0.0106952     11  0.49198 0.71658 0.053626
```

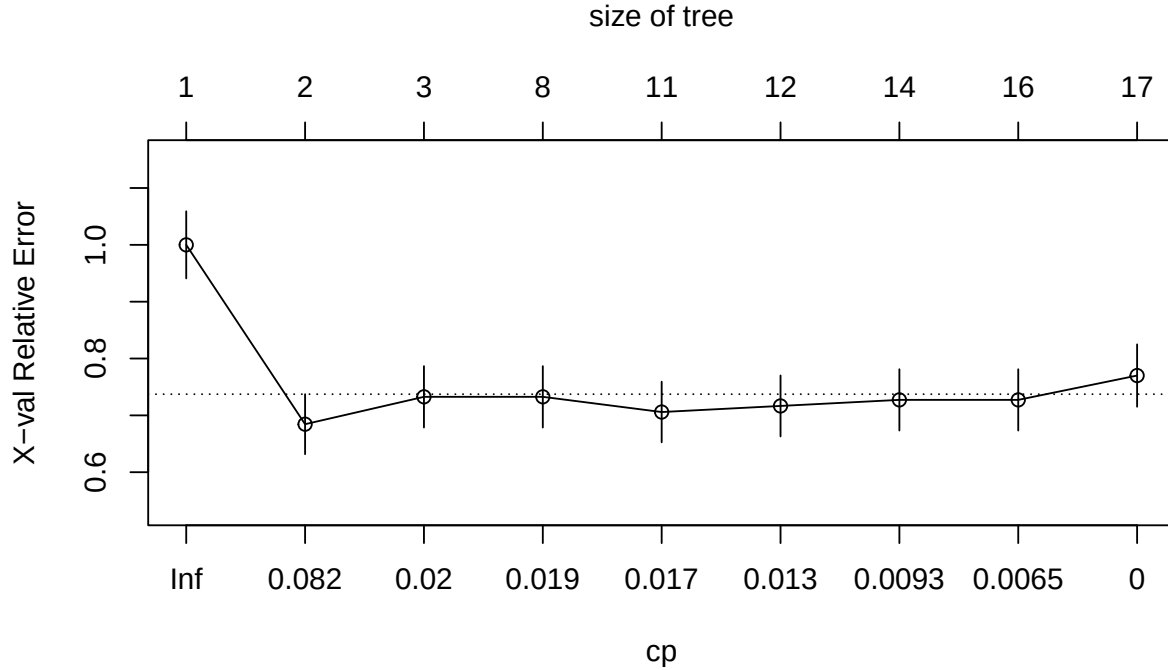



Figure 1: Cross-validation error for different complexity parameter (CP) values.

```
## 7 0.0080214    13  0.47059 0.72727 0.053891
## 8 0.0053476    15  0.45455 0.72727 0.053891
## 9 0.0000000    16  0.44920 0.77005 0.054897
```

In the `rpart` implementation, the complexity parameter is denoted by CP and controls the trade-off between tree size and predictive performance. Larger values of CP impose a stronger penalty on additional splits, resulting in smaller trees. The `printcp()` table reports the cross-validation error (`xerror`) for a sequence of candidate subtrees, while the `plotcp()` function provides a graphical display of the cross-validation error for each candidate subtree.

The optimal value of CP is selected using cross-validation. Starting from the fully grown tree, CART generates a sequence of increasingly smaller subtrees. The predictive performance of each subtree is estimated by cross-validation, and the final tree is selected according to the smallest cross-validation error (`xerror`).

The optimal complexity parameter is selected as:

$$CP^* = \arg \min_{CP} \{xerror(CP)\}$$

For the diabetes example, the optimal complexity parameter is obtained from the row corresponding to the smallest value of `xerror`: $CP^* = 0.02139$.

This value of CP is used to prune the original tree and obtain the final CART model.

```
optimal_cp <- cart_full$cptable[which.min(cart_full$cptable[, "xerror"]), "CP"]
optimal_cp
```

```
## [1] 0.02139037
```

Pruning therefore represents an application of the model validation principles introduced previously: rather than selecting the tree with the smallest training error, we select the tree expected to perform best on future observations.

```

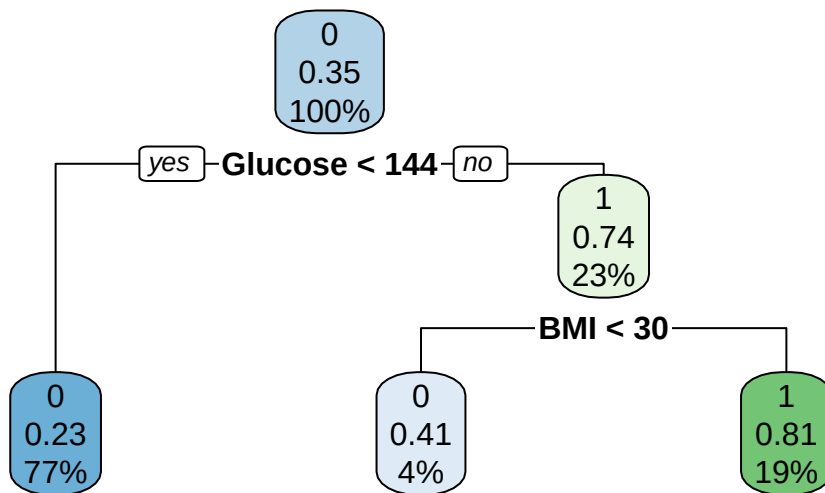
set.seed(333)

# pruned fit
cart_pruned <- rpart(
  Outcome ~ .,
  data = train_data,
  method = "class",
  control = rpart.control(cp = optimal_cp)
)

# or simply
# cart_pruned2 <- prune(cart_full, cp=optimal_cp)

rpart.plot(cart_pruned)

```



```

#or
#plot(cart_pruned)
#text(cart_pruned, xpd=T, cex=0.6)

# Prediction
pred_cart_pruned <- predict(cart_pruned, newdata = test_data, type = "class")

library(caret)
confusionMatrix(pred_cart_pruned, as.factor(test_data$Outcome), positive = "1")

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 137  54
##           1   13  27
##
##           Accuracy : 0.71
##           95% CI   : (0.6468, 0.7676)
##           No Information Rate : 0.6494

```

```
##      P-Value [Acc > NIR] : 0.02999
##
##              Kappa : 0.2792
##
## Mcnemar's Test P-Value : 1.025e-06
##
##      Sensitivity : 0.3333
##      Specificity : 0.9133
##      Pos Pred Value : 0.6750
##      Neg Pred Value : 0.7173
##      Prevalence : 0.3506
##      Detection Rate : 0.1169
##      Detection Prevalence : 0.1732
##      Balanced Accuracy : 0.6233
##
##      'Positive' Class : 1
##
```

```
# Accuracy
accuracy_cart_pruned <- mean(pred_cart_pruned == test_data$Outcome)

data.frame(Model = c("Unpruned CART", "Pruned CART"),
           Accuracy = c(accuracy_cart_full, accuracy_cart_pruned)
)
```

```
##      Model  Accuracy
## 1 Unpruned CART 0.7229437
## 2   Pruned CART 0.7099567
```

Although the pruned CART does not achieve a higher test accuracy for this particular train-test split, it substantially reduces model complexity while maintaining comparable predictive performance.

$$Accuracy_{unpruned} = 0.723, \quad Accuracy_{pruned} = 0.710$$

This illustrates an important principle in statistical modelling:

A simpler model with similar predictive ability is often preferred.

The pruned tree sacrifices a small amount of flexibility in exchange for a simpler decision rule, easier interpretation, reduced model complexity, and lower sensitivity to noise in the training data.

Therefore, pruning is not only a method for improving prediction accuracy, but also a method for controlling model complexity and improving the model's ability to generalize to new data.

3. Bagging: Bootstrap Aggregation

A major limitation of individual CART models is their instability. A small change in the training dataset can alter the sequence of splits and produce a different final tree.

This occurs because decision trees are high-variance models. They are flexible enough to represent nonlinear relationships and interactions, but their predictions can be strongly influenced by the particular observations used for training.

For example, in environmental applications, a CART model predicting flood magnitude from catchment characteristics may choose different splitting rules depending on whether a few extreme flood events are included or excluded from the training data.

One strategy to reduce this instability is to average many different trees. This approach is known as **bootstrap aggregation**, or **bagging**.

The main idea is:

$$\boxed{\text{Bagging} = \text{Bootstrap samples} + \text{Many CART models} + \text{Average prediction}}$$

3.1 Bootstrap samples

Suppose we have an environmental dataset consisting of n observations:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}.$$

A bootstrap sample is created by repeatedly sampling n observations with replacement from the original dataset.

The resulting bootstrap datasets are

$$D_1^*, D_2^*, \dots, D_B^*,$$

where B is the number of bootstrap replicates.

A separate CART model is fitted to each bootstrap dataset:

$$\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_B(x).$$

Because each bootstrap sample contains slightly different observations, the resulting trees are different versions of the same underlying model.

3.2 Combining bootstrap predictions

For a new observation with predictors x , each tree produces a prediction:

$$\hat{f}_b(x).$$

For regression problems, bagging combines these predictions by averaging:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

For classification problems, predictions are combined using majority voting:

$$\hat{y}_{\text{bag}} = \text{majority}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B\}.$$

Thus, bagging replaces one unstable tree with an ensemble of trees.

3.3 Why does bagging improve prediction?

The expected prediction error can be decomposed as:

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2.$$

where:

- Bias^2 represents systematic error;

- Variance represents sensitivity to the training data;
- σ^2 represents irreducible observation error.

Individual CART models usually have:

low bias + high variance.

Bagging reduces the variance component by averaging many independently fitted trees:

Variance ↓

while the bias remains approximately unchanged.

Therefore, bagging is particularly effective for flexible learners such as CART.

3.4 Bagging in R

Bagging can be implemented using the `randomForest` package by setting the number of predictors considered at each split equal to the total number of available predictors.

This produces a bagged ensemble of CART models. Random Forest extends this idea by introducing additional randomness through predictor selection at each split.

For the diabetes dataset:

```
library(randomForest)

set.seed(123)
bag_model <- randomForest(
  Outcome ~ .,
  data = train_data,
  mtry = ncol(train_data)-1,
  ntree = 500
)
bag_model

##
## Call:
## randomForest(formula = Outcome ~ ., data = train_data, mtry = ncol(train_data) - 1, ntree = 500)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 8
##
##              OOB estimate of  error rate: 25.14%
## Confusion matrix:
##      0   1 class.error
## 0 295  55  0.1571429
## 1  80 107  0.4278075
```

Here, $B = 500$ trees are fitted. For each observation, the final class prediction is obtained by majority voting across all trees. For regression problems, the final prediction would instead be the average prediction.

In practice, the number of bootstrap trees B is chosen sufficiently large so that the ensemble prediction becomes stable. Increasing B reduces the Monte Carlo variability of the bagging procedure, although the improvement after a sufficiently large number of trees is usually small.

The fitted bagging model therefore represents an ensemble of CART models, where each tree is trained on a different bootstrap sample and the final prediction is obtained by averaging across the ensemble.

The performance of the bagged model can be compared with the original CART model using the independent test dataset.

```
# Prediction from bagging model
pred_bag <- predict(bag_model, newdata = test_data)

# Accuracy
accuracy_bag <- mean(pred_bag == test_data$Outcome)

data.frame(Model = c("Single CART", "Bagging CART"),
  Accuracy = c(accuracy_cart_full, accuracy_bag))

##           Model Accuracy
## 1 Single CART 0.7229437
## 2 Bagging CART 0.7489177
```

The performance of the single CART and bagged CART models can be compared using the independent test dataset.

For this split of the data, $Accuracy_{CART} = 0.723$ whereas $Accuracy_{Bagging} = 0.749$.

The improvement is relatively small, but this is expected. Bagging does not directly increase the flexibility of the individual trees. Instead, it reduces the variability associated with the training sample by averaging many trees.

The main advantage of bagging is therefore improved stability:

$$\text{Variance}_{\text{bagging}} < \text{Variance}_{\text{single CART}}.$$

A single CART may change substantially if a few observations are modified, whereas a bagged model is less sensitive because the final prediction is based on many bootstrap samples.

3.5 From Bagging to Random Forest

Although bagging improves the stability of CART models, all trees in the ensemble are constructed using the same set of predictors at each split.

Consequently, if one or two predictors are particularly strong, they may be selected repeatedly across many trees. This causes the individual trees to remain correlated.

However, averaging works best when the individual models are both accurate and different from each other. Highly correlated trees provide less additional information when combined.

Random Forest addresses this limitation by introducing an additional source of randomness. At each split, only a random subset of predictors is considered.

Therefore:

$\text{Random Forest} = \text{Bagging} + \text{Random predictor selection}$

The objective is not only to reduce the variance of individual trees, but also to reduce the correlation between trees in the ensemble.

4. Random Forest

Bagging reduces the variance of CART models by averaging predictions from many bootstrap samples. However, bagged trees have an important limitation: the individual trees are often highly correlated.

This occurs because every tree considers all available predictors when searching for the best split. If one predictor is particularly strong, many bootstrap trees will repeatedly select the same predictor near the top of the tree. Consequently, the resulting trees may become similar to each other.

Averaging highly correlated trees provides less variance reduction because the errors of the individual models are not independent.

Random Forests address this limitation by introducing an additional source of randomness: at each split, only a random subset of predictors is considered.

The resulting model is an ensemble of decorrelated CART models.

4.1 Random predictor selection

Suppose the dataset contains p predictor variables:

$$X = \{X_1, X_2, \dots, X_p\}.$$

In ordinary CART and bagging, every split considers all p predictors.

For Random Forests, a random subset of size m is selected at each split:

$$m < p.$$

The best split is then chosen only among these randomly selected predictors.

The procedure is repeated independently at every node of every tree.

Therefore, two trees grown from different bootstrap samples will also differ because they examine different predictor subsets.

4.2 Random Forest algorithm

A Random Forest with B trees is constructed as follows:

1. Generate a bootstrap sample from the original dataset.
2. Grow a CART tree using the bootstrap sample.
3. At every node, randomly select m predictors from the available predictors.
4. Find the best split among these selected predictors.
5. Continue splitting until the tree is fully grown (Random Forest trees are generally not pruned).
6. Repeat steps 1–5 for B trees.

The final regression prediction is obtained by averaging the individual tree predictions:

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

For classification problems, the final prediction is obtained by majority vote:

$$\hat{y}_{\text{RF}} = \text{mode}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B\}.$$

4.3 Difference between Bagging and Random Forests

The additional randomization decreases the correlation between trees, which improves the efficiency of averaging.

The main differences are:

Method	Predictors at split	Tree size
CART	All predictors	Pruned
Bagging	All predictors	Usually large
Random Forest	Random subset	Usually large

Unlike CART, Random Forest trees are generally grown fully and are not pruned.

The reason is that averaging many high-variance trees is more effective than attempting to optimize a single tree.

4.4 Out-of-bag error estimation

Because each tree is fitted using a bootstrap sample, approximately one third of observations are not included in that sample.

These observations are called **out-of-bag (OOB) observations**.

For observation i , prediction can be obtained only from trees where observation i was not included in the bootstrap sample:

$$\hat{y}_i^{OOB} = \frac{1}{B_i} \sum_{b \in O_i} \hat{f}_b(x_i),$$

where:

- O_i is the set of trees for which observation i is out-of-bag,
- B_i is the number of such trees.

The OOB prediction error provides an internal estimate of model performance without requiring an independent test dataset.

For classification problems, the OOB error is measured as the proportion of incorrect OOB predictions:

$$Error_{OOB} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i^{OOB}),$$

where $I(\cdot)$ is an indicator function that equals 1 when the prediction is incorrect and 0 otherwise.

For regression problems, the OOB error is usually measured using the root mean squared error (RMSE):

$$RMSE_{OOB} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{OOB})^2}.$$

4.5 Environmental interpretation

Consider predicting flood frequency from catchment characteristics:

$$Y = \text{flood frequency}$$

with predictors:

$$X = \{\text{rainfall, catchment area}\}.$$

A single CART might split first on rainfall and then on catchment area. However, small changes in the available catchments may change these decisions.

A Random Forest creates many alternative hydrological models:

$$f_1(x), f_2(x), \dots, f_B(x),$$

each representing a different possible relationship between rainfall, area, and flood frequency.

The final prediction is the average of these models, resulting in a more stable estimate.

4.6 Random Forest in R

```
library(randomForest)

set.seed(123)
rf_model <- randomForest(
  Outcome ~ .,
  data = train_data,
  mtry = 3,
  ntree = 500,
  importance = TRUE
)

# Prediction from rf_model
pred_rf <- predict( rf_model, newdata = test_data)

# Accuracy
accuracy_rf <- mean(pred_rf == test_data$Outcome)

library(ranger)
set.seed(123)

rf_model2 <- ranger(Outcome ~ ., data = train_data, num.trees = 500, mtry = 3,
  importance = "impurity")
#rf_model2
accuracy_rf

## [1] 0.7489177
```

The predictive performance of the three tree-based models can be compared using the independent test dataset.

For this split of the data:

- $Accuracy_{CART} = 0.723$,
- $Accuracy_{Bagging} = 0.749$, and
- $Accuracy_{RandomForest} = 0.749$.

For the ensemble approach, bagging reduces the high variance of individual trees by averaging many bootstrap models.

Random Forest provides a further improvement by reducing the correlation between trees through random predictor selection at each split.

However, the improvement from Random Forest over bagging is often smaller than the improvement from CART to bagging, because bagging already captures much of the available variance reduction.

4.7 Variable importance

Random Forests provide measures of predictor importance. Two commonly used measures are mean decrease in accuracy and mean decrease in Gini impurity.

The mean decrease in accuracy measures (does this variable improve prediction?) how much predictive performance is lost when a predictor is randomly permuted:

$$Importance_j = Accuracy_{original} - Accuracy_{permuted(j)}$$

Therefore, it reflects the contribution of a variable to predictive accuracy.

The mean decrease in Gini measures (does this variable help create purer tree nodes?) how much a predictor contributes to reducing node impurity:

$$MDI_j = \frac{1}{B} \sum_{b=1}^B \sum_t \Delta G_{j,t,b}.$$

where:

- j denotes predictor variable j ;
- t denotes a split involving predictor j ;
- ΔG represents the reduction in Gini impurity.

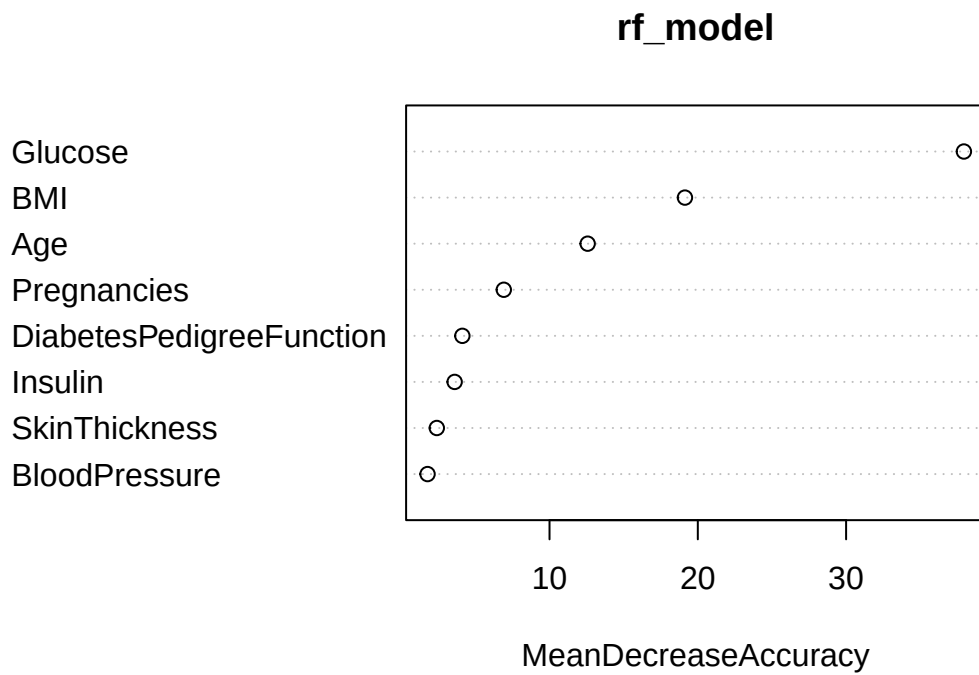
Therefore, it reflects how strongly a variable contributes to creating homogeneous daughter nodes during tree construction.

For Gini importance, variables that repeatedly create homogeneous daughter nodes receive higher importance scores.

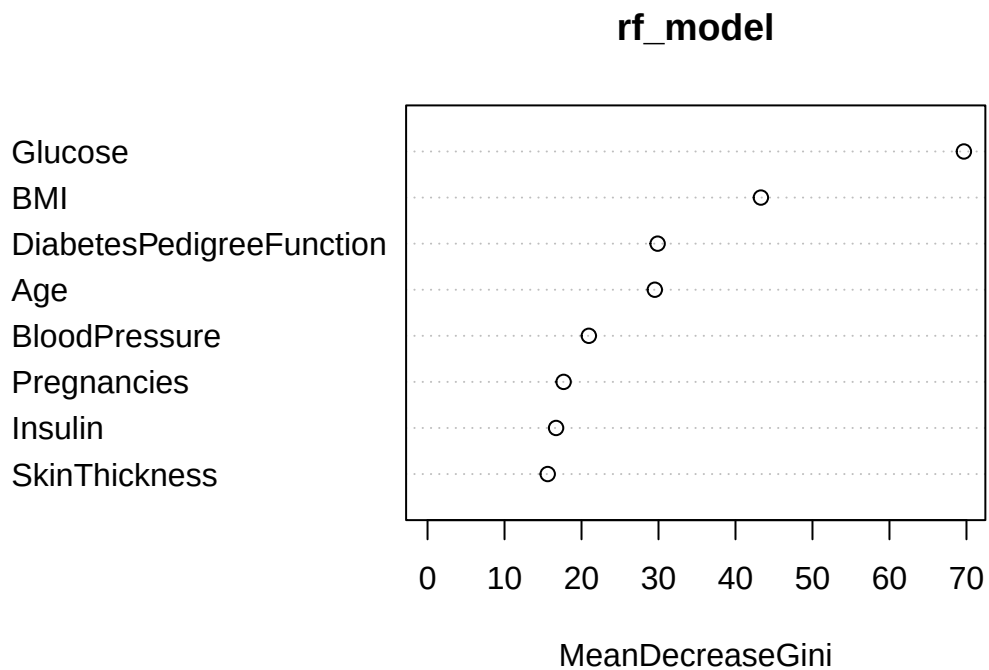
```
randomForest::importance(rf_model)
```

```
##              0              1 MeanDecreaseAccuracy
## Pregnancies      7.722092  1.342368              6.916511
## Glucose          28.915312 26.553435              37.942921
## BloodPressure    2.016836  0.446786              1.775156
## SkinThickness    5.954611 -4.514579              2.398816
## Insulin          7.477111 -3.784710              3.601975
## BMI             10.447792 17.820779              19.130649
## DiabetesPedigreeFunction 3.128643  2.789752              4.121613
## Age             9.484201  6.706925              12.571679
##              MeanDecreaseGini
## Pregnancies      17.67261
## Glucose          69.66731
## BloodPressure    20.94986
## SkinThickness    15.60555
## Insulin          16.69588
## BMI             43.29779
## DiabetesPedigreeFunction 29.88431
## Age             29.51761
```

```
varImpPlot(rf_model, type = 1)
```



```
varImpPlot(rf_model, type = 2)
```



```
#varImpPlot(rf_model)
```

In this dataset, glucose concentration generally receives high importance because it provides strong separation

between diabetic and non-diabetic observations.

4.8 Examining individual trees

A Random Forest is an ensemble consisting of many individual CART models:

$$\mathcal{F} = \{T_1, T_2, \dots, T_B\},$$

where B is the number of trees in the forest.

Although the final prediction is obtained by combining all trees, individual trees can still be extracted to examine their structure.

In R, the structure of tree k can be obtained using:

```
tree_1 <- getTree(rf_model, k = 1, labelVar = TRUE)
head(tree_1)

tree_2 <- getTree(rf_model, k = 2, labelVar = TRUE)
head(tree_2)

tree_3 <- getTree(rf_model, k = 3, labelVar = TRUE)
head(tree_3)

# treeInfo(rf_model2, tree=1) is used for ranger() fit
```

$tree_1$, $tree_2$, and $tree_3$ represent the first three individual CART trees contained inside the fitted Random Forest model. Each row represents a node rather than an observation. For instance, for $tree_1$, node 1 (the root node) splits the observations according to $Age \leq 28.5$.

- If $Age \leq 28.5$, observations move to node 2 (left daughter),
- If $Age > 28.5$, observations move to node 3 (right daughter).

The tree then continues splitting these daughter nodes. For instance, observations reaching node 2 are further split using $DiabetesPedigreeFunction \leq 0.454$. If true $\rightarrow$ move to node 4, otherwise move to node 5.

For each tree structure:

- Rows with **status=-1** represent terminal nodes, where the final class prediction is assigned (`randomForest::getTree()`).
- Rows where **terminal = TRUE** (`ranger::treeInfo()`) represent terminal nodes, where the final class prediction is assigned.

Although an individual tree can be examined, the Random Forest prediction does not depend on this single tree. Instead, predictions from all trees are combined:

$$\hat{y}_{RF} = \text{majority}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B\}.$$

5. Weighted Learning and the Principle Behind Boosting

In ordinary machine learning models, all observations usually contribute equally to the estimation of the model. However, some observations may be more important than others during the learning process.

For example, in environmental applications, extreme events such as rare floods or droughts may require greater attention because they are often underrepresented in the available data and may be difficult for models to predict accurately.

One way to control the influence of observations is to assign each observation a weight. Weighted learning allows some observations to contribute more strongly than others during model fitting.

The general weighted loss function can be written as:

$$L_w = \sum_{i=1}^n w_i L(y_i, f(x_i)),$$

where:

- $L(y_i, f(x_i))$ is the loss associated with prediction error;
- w_i is the weight assigned to observation i ;
- larger weights increase the influence of an observation during training.

When all observations have equal weights:

$$w_1 = w_2 = \dots = w_n = 1,$$

the model reduces to ordinary unweighted learning.

5.1 Weighted CART models

CART models can also incorporate observation weights.

For classification trees, the splitting criterion is based on class impurity. The standard Gini impurity is:

$$G = 1 - \sum_{k=1}^K p_k^2,$$

where p_k is the proportion of observations belonging to class k .

When observation weights are included, the class proportions are calculated using weighted observations:

$$p_k^{(w)} = \frac{\sum_{i=1}^n w_i I(y_i = k)}{\sum_{i=1}^n w_i},$$

where:

- w_i is the weight assigned to observation i ;
- $I(y_i = k)$ is an indicator function equal to 1 when observation i belongs to class k , and 0 otherwise.

The weighted Gini impurity is therefore:

$$G_w = 1 - \sum_{k=1}^K \left(p_k^{(w)}\right)^2.$$

The weighted impurity measure modifies the split selection process because observations with larger weights have a greater influence on the class composition of each node.

For the diabetes example:

$$Y \in \{0, 1\},$$

we can assign greater importance to one class:

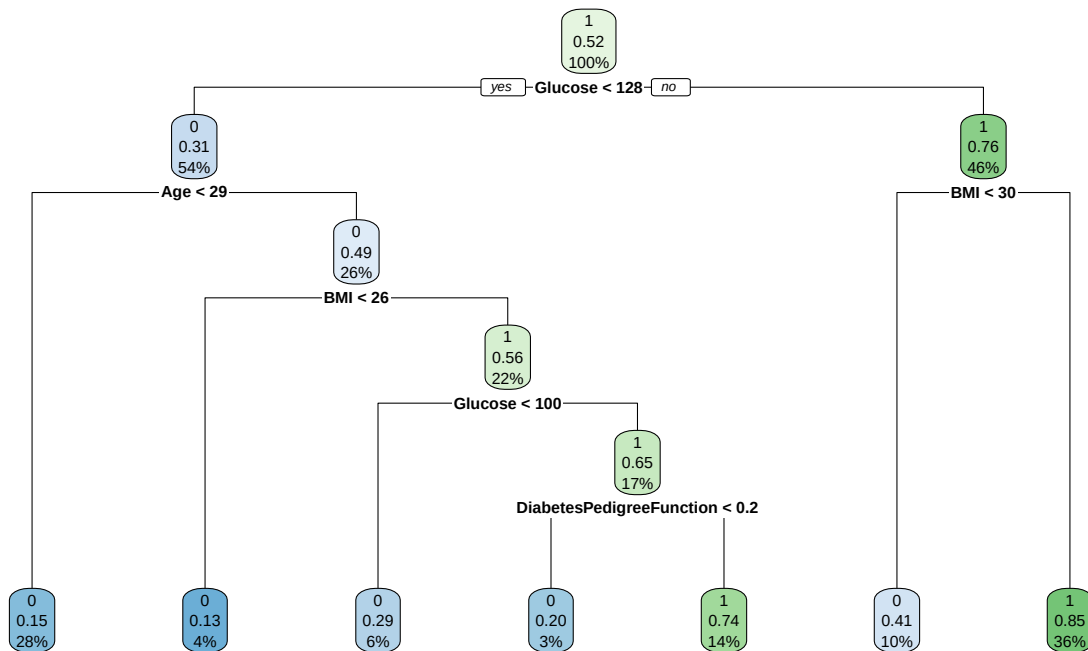
```

library(rpart)
set.seed(123)

weighted_tree <- rpart(
  Outcome ~ .,
  data = train_data,
  method = "class",
  weights = ifelse(train_data$Outcome == 1, 2, 1),
  control = rpart.control(
    cp = optimal_cp
  )
)

rpart.plot(weighted_tree)

```



```
# Prediction from weighted CART
```

```
pred_weighted <- predict(weighted_tree, newdata = test_data, type = "class")
```

```
# Accuracy
```

```
accuracy_weighted <- mean(pred_weighted == test_data$Outcome)
accuracy_weighted
```

```
## [1] 0.7445887
```

The purpose of weighting is not necessarily to increase overall prediction accuracy. Instead, weighting changes the relative importance of observations during model fitting.

In this example, diabetic cases (*Outcome* = 1) were assigned a larger weight. Therefore, these observations contribute more strongly to the impurity calculation during tree construction. This can change the selected

splits and, in this example, improves the ability of the model to identify positive cases.

Compared with the standard Random Forest model, the weighted CART model achieved a similar overall accuracy:

$$Accuracy_{Weighted\ CART} = 0.745$$

compared with:

$$Accuracy_{Random\ Forest} = 0.749.$$

However, the weighted model increased sensitivity:

$$Sensitivity_{Weighted\ CART} = 0.704$$

compared with:

$$Sensitivity_{Random\ Forest} = 0.568.$$

This demonstrates that weighting can modify the trade-off between different types of prediction errors. In classification problems where false negatives are more costly than false positives, increasing the weight of the positive class may provide a more appropriate decision rule.

This changes the fitted tree because the splitting process is based on weighted class proportions rather than ordinary class proportions. Consequently, splits that better separate the higher-weight observations may be selected instead of the splits chosen by an unweighted CART model.

However, this is a fixed weighting scheme. The weights are specified before model fitting and remain unchanged throughout the learning process.

Boosting extends this idea by sequentially adjusting the learning process based on previous model errors. In algorithms such as AdaBoost, this is achieved by updating observation weights, while other boosting methods use alternative mechanisms based on residual correction.

5.2 Comparison with bagging and Random Forests

The main differences between these tree-based methods can be summarized as:

Table 1: Comparison of tree-based methods introduced in this chapter.

Method	Main principle	Main objective
CART	Single recursive decision tree	Interpretable prediction model
Weighted CART	CART with observation-specific weights	Increase influence of selected observations
Bagging	Bootstrap averaging of multiple CARTs	Reduce variance through averaging
Random Forest	Bagging with random predictor selection	Reduce variance and tree correlation

The main distinction is that Weighted CART modifies the construction of a single tree by changing the relative importance of observations. In contrast, Bagging and Random Forests improve prediction by combining multiple trees.

Weighted CART therefore introduces an important concept: modifying the learning process through observation importance. Boosting extends this idea by updating the importance of observations during the learning process.

```
##           Model  Accuracy
## 1 Unpruned CART 0.7229437
## 2   Pruned CART 0.7099567
## 3       Bagging 0.7489177
## 4 Random Forest 0.7489177
## 5 Weighted CART 0.7445887
```

5.3 From weighted learning to boosting

The main idea behind boosting is that some observations are more difficult to predict than others.

A first weak learner is fitted to the data:

$$f_1(x).$$

After this model is evaluated, the learning process is adjusted so that future learners focus more strongly on observations where the current model performs poorly. In algorithms such as AdaBoost, this adjustment is achieved through updating observation weights, whereas other boosting methods use alternative mechanisms such as residual correction.

A new weak learner is then fitted to improve the remaining prediction errors:

$$f_2(x).$$

This process is repeated sequentially, where each new learner attempts to improve the limitations of the previous learners:

Fit model → Identify remaining errors → Adjust the learning process → Fit next model

For classification problems, the final prediction is obtained by combining the individual learners and transforming the resulting score into a class prediction.

Therefore, unlike bagging and Random Forests, where trees are fitted independently and combined through averaging, boosting creates a sequence of dependent models. Each new model attempts to correct the errors made by the previous models.

5.4 Summary

Weighted learning provides the foundation for understanding boosting.

The key concept is:

Change observation importance → Focus on difficult cases → Sequentially improve the model

Boosting applies this principle repeatedly, creating an ensemble where each new learner improves upon the limitations of previous learners.

6. Boosting

Bagging and Random Forests improve prediction by fitting many models independently and combining their predictions. In contrast, **boosting** is a sequential learning procedure in which each new model is fitted to address the remaining errors of the current ensemble.

The central idea of boosting is that a collection of weak learners can be combined to create a strong predictive model. Rather than building many independent trees, boosting constructs a sequence of models where each new learner contributes additional information and improves the current prediction.

A weak learner is a simple model with limited predictive ability that captures some structure in the data but does not fully describe the response. For CART-based boosting algorithms, weak learners are typically shallow decision trees with only a small number of splits.

By combining many such weak learners, boosting can produce a flexible model with substantially improved predictive performance.

6.1 Sequential learning

Suppose the response variable is Y and the predictors are represented by:

$$\mathbf{x} = (x_1, x_2, \dots, x_p).$$

The aim is to estimate an unknown function:

$$Y = f(\mathbf{x}) + \epsilon.$$

Boosting constructs an additive model consisting of many simple learners:

$$F_M(\mathbf{x}) = \sum_{m=1}^M \gamma_m h_m(\mathbf{x}),$$

where:

- $h_m(\mathbf{x})$ is the m -th weak learner,
- γ_m is the contribution of that learner,
- M is the total number of learners.

Unlike bagging, the learners are not fitted independently. Each learner depends on the predictions of all previous learners.

6.2 The boosting algorithm

Boosting constructs the final model by adding weak learners sequentially. Each new learner is fitted to improve the predictions produced by the current ensemble.

The main steps of the boosting algorithm are:

- initialize the model with $F_0(\mathbf{x})$;
- measure the remaining prediction errors;
- fit a new weak learner $h_m(\mathbf{x})$;
- update the current model $F_m(\mathbf{x})$;
- repeat the process and combine all learners into a final additive model.

The algorithm begins with an initial prediction:

$$F_0(\mathbf{x}).$$

At each iteration, the current model is evaluated and the remaining prediction errors are measured. For regression problems with squared-error loss, these errors are represented by residuals:

$$r_i^{(m)} = y_i - F_{m-1}(\mathbf{x}_i),$$

where $F_{m-1}(\mathbf{x}_i)$ is the prediction from the current model and $r_i^{(m)}$ represents the error that remains to be corrected.

A new weak learner is then fitted to these remaining errors:

$$h_m(\mathbf{x}) \approx r^{(m)}.$$

The new learner is added to the existing model:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \gamma_m h_m(\mathbf{x}),$$

where γ_m controls the contribution of the new learner.

This process is repeated for $m = 1, \dots, M$. Each additional learner focuses on reducing the remaining errors of the current ensemble.

After M iterations, the final boosted model is:

$$F_M(\mathbf{x}) = F_0(\mathbf{x}) + \sum_{m=1}^M \gamma_m h_m(\mathbf{x}).$$

The final model is therefore an additive combination of many weak learners, where each learner contributes a small improvement to the current prediction. The strength of boosting comes from gradually improving the model through this sequential learning process.

6.3 Learning rate

In practice, boosting algorithms introduce a shrinkage parameter, also called the learning rate:

$$0 < \nu \leq 1.$$

The update equation becomes:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \gamma_m h_m(\mathbf{x}).$$

The learning rate controls the contribution of each new weak learner to the current model. A smaller value of ν reduces the step size of each update, meaning that individual learners have less influence on the final prediction.

This has two important consequences:

- more weak learners are required to achieve the same level of model improvement;
- the model often generalizes better because learning occurs more gradually, which can reduce the risk of overfitting.

Therefore, boosting models are commonly fitted using many simple learners, often shallow trees, combined with a small learning rate.

6.4 Boosting versus bagging

Although both methods combine multiple learners, the way the learners are constructed is different.

For bagging, the final prediction is obtained by averaging independently fitted models:

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x}),$$

where each model is fitted independently.

For boosting, learners are fitted sequentially, with each new learner improving the current model:

$$F_M(\mathbf{x}) = \sum_{m=1}^M \gamma_m h_m(\mathbf{x}),$$

The main differences can be summarized as:

Table 2: Comparison of bagging and boosting algorithms.

Feature	Bagging	Boosting
Training	Independent	Sequential
Main objective	Reduce variance	Reduce bias and prediction error
Tree type	Large CARTs	Usually shallow CARTs
Observation weights	Equal weights	Updated sequentially during learning

6.5 Boosting example in R

A simple implementation using the `gbm` package:

```
library(gbm)
set.seed(123)

boost_model <- gbm(
  as.numeric(as.character(Outcome)) ~ .,
  data = train_data,
  distribution = "bernoulli",
  n.trees = 500,
  interaction.depth = 2,
  shrinkage = 0.01
)
# Predict probabilities for the test data
pred_prob_boost <- predict(boost_model, newdata = test_data, n.trees = 500, type = "response")

# Convert probabilities to class predictions
pred_boost <- ifelse(pred_prob_boost > 0.5, 1, 0)

# Confusion matrix
confusionMatrix(factor(pred_boost), (test_data$Outcome), positive = "1")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 129  41
##           1  21  40
##
##           Accuracy : 0.7316
##           95% CI : (0.6696, 0.7876)
##       No Information Rate : 0.6494
##       P-Value [Acc > NIR] : 0.00471
##
##           Kappa : 0.3751
##
##  McNemar's Test P-Value : 0.01582
##
##           Sensitivity : 0.4938
##           Specificity : 0.8600
##       Pos Pred Value : 0.6557
##       Neg Pred Value : 0.7588
##           Prevalence : 0.3506
##       Detection Rate : 0.1732
##       Detection Prevalence : 0.2641
```

```
##          Balanced Accuracy : 0.6769
##
##          'Positive' Class : 1
##
# Accuracy
accuracy_boost <- mean( pred_boost == test_data$Outcome)
accuracy_boost

## [1] 0.7316017
```

The main parameters controlling a boosting model are the number of weak learners, the learning rate, and the complexity of the individual trees.

The number of boosting iterations ($M = \mathbf{n.trees}$), represents the total number of weak learners included in the final ensemble.

The contribution of each learner is controlled by the learning rate ($\nu = \mathbf{shrinkage}$), which determines the step size used when adding each new learner to the current model.

The complexity of each individual tree is controlled by $d = \mathbf{interaction.depth}$, which represents the maximum depth of each weak learner.

Larger values of d allow more complex interactions between predictors, whereas smaller values produce simpler trees. In boosting, shallow trees are commonly used because the final model is obtained by combining many sequential learners, with each learner providing a small correction to the current model.

The predictive performance of the fitted models can be compared using test accuracy:

Table 3: Comparison of classification model accuracy.

Model	Accuracy
Unpruned CART	0.723
Pruned CART	0.710
Bagging	0.749
Random Forest	0.749
Weighted CART	0.745
Boosting	0.732

7. Extreme Gradient Boosting (XGBoost)

Boosting methods have become among the most successful machine-learning approaches for structured or tabular datasets. In many tabular-data problems, tree-based boosting methods can achieve predictive performance comparable to or better than neural networks. A particularly influential extension is **Extreme Gradient Boosting (XGBoost)**, which is an optimized implementation of gradient boosting based on regression trees.

XGBoost has achieved excellent predictive performance in many environmental applications, including species distribution modelling, climate prediction, remote sensing, and hydrological modelling.

The success of XGBoost results from several improvements over traditional boosting algorithms:

- gradient-based optimization of the loss function;
- regularization to reduce overfitting;
- efficient tree construction;
- shrinkage through the learning rate;
- subsampling of observations and predictors.

7.1 Gradient boosting formulation

Traditional boosting can be viewed as sequentially correcting previous prediction errors. Gradient boosting generalizes this idea by fitting new learners to the negative gradient of a loss function.

The objective is to minimize:

$$\mathcal{L}(F) = \sum_{i=1}^n L(y_i, F(\mathbf{x}_i)),$$

where:

- L is the loss function,
- y_i is the observed response,
- $F(\mathbf{x}_i)$ is the current prediction.

At iteration m , the pseudo-residuals are calculated as:

$$r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$$

The pseudo-residual represents the direction in which the current prediction should be improved to reduce the loss function.

A new regression tree is then fitted to these pseudo-residuals:

$$h_m(\mathbf{x}) \approx r_m$$

The model is updated:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x}),$$

where ν is the learning rate.

7.2 Regularization in XGBoost

A key difference between XGBoost and traditional boosting algorithms is the explicit use of regularization.

The objective function can be written as:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k),$$

where:

- $l(y_i, \hat{y}_i)$ measures prediction error,
- f_k represents an individual regression tree,
- $\Omega(f_k)$ penalizes tree complexity.

The regularization term is commonly expressed as:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

where:

- T is the number of terminal nodes,
- w_j is the prediction value of terminal node j ,
- γ controls the penalty for additional splits,

- λ controls the penalty on leaf weights.

Therefore, XGBoost does not simply minimize training error; it balances predictive accuracy with model complexity.

7.3 Why XGBoost performs well for environmental data

Environmental datasets are often characterized by:

- nonlinear relationships;
- interactions between predictors;
- heterogeneous predictor variables;
- missing values;
- complex response surfaces.

For example, flood risk may depend on interactions between:

$$\text{rainfall} \times \text{catchment area} \times \text{land characteristics}.$$

A linear model assumes that effects are additive:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon,$$

whereas a tree ensemble can represent complex interactions:

$$Y = f(X_1, X_2, \dots, X_p).$$

Because trees partition the predictor space recursively, they can naturally capture thresholds and nonlinear responses.

7.4 XGBoost versus neural networks

For structured environmental datasets, XGBoost and related tree-based methods are often among the strongest predictive approaches.

Neural networks are highly flexible models, but they generally perform best when large amounts of high-dimensional data are available, such as images, text, or sensor streams.

For many environmental tabular datasets,

$$\{\text{climate variables, soil properties, land use, topography}\},$$

tree-based boosting methods can provide excellent predictive accuracy with relatively limited data preprocessing.

However, predictive performance should always be evaluated using appropriate validation strategies, such as cross-validation or spatial block cross-validation, because environmental observations are often not independent.

7.5 XGBoost example in R

The `xgboost` package implements gradient boosting using regression trees.

```

library(xgboost)
set.seed(123)

X_train <- as.matrix(train_data[, !names(train_data) %in% "Outcome"])
X_test  <- as.matrix(test_data[, !names(test_data) %in% "Outcome"])

y <- train_data$Outcome

xgb_model <- xgboost(
  data = X_train,
  label = y,
  nrounds = 500,
  max_depth = 2,
  eta = 0.01,
  objective="binary:logistic", # multi:softprob
)
# Predict probabilities for the test data
pred_xgb <- predict(xgb_model, newdata = X_test, n.trees = 500, type = "class")

# Confusion matrix
confusionMatrix(pred_xgb, test_data$Outcome, positive = "1")

# Accuracy
accuracy_xgb <- mean(pred_xgb == test_data$Outcome)
accuracy_xgb

```

The main parameters controlling an XGBoost model are the number of boosting iterations, the complexity of individual trees, and the learning rate.

The number of boosting iterations ($M = \text{nrounds}$), represents the total number of regression trees included in the ensemble.

The complexity of each individual tree is controlled by $d = \text{max_depth}$, which represents the maximum depth of each regression tree.

The contribution of each tree to the ensemble is controlled by the learning rate ($\eta = \text{eta}$).

The model update can therefore be expressed as:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}),$$

where $h_m(\mathbf{x})$ is the m -th regression tree.

Smaller values of η reduce the contribution of individual trees and therefore require a larger number of boosting iterations M . However, smaller learning rates often improve generalization by reducing the risk of overfitting.

8. Calibration of Tree-Based Prediction Models

Tree-based classification models such as CART, Random Forests, and boosting algorithms are primarily optimized to improve classification performance. Although these models can produce predicted probabilities, these probabilities may not always correspond to reliable estimates of event likelihood.

For binary classification problems:

$$Y \in \{0, 1\},$$

a model produces a predicted probability:

$$\hat{p} = P(Y = 1).$$

Calibration describes the relationship between predicted probabilities and observed event frequencies.

For example, if a model predicts:

$$\hat{p} = 0.7,$$

for a group of observations, a well-calibrated model should observe the event in approximately 70% of those cases.

However, tree-based models are mainly optimized for separating classes rather than directly estimating accurate probabilities. As a result, predicted probabilities may be too extreme or too conservative. For example, predictions close to 0 or 1 may indicate high model confidence but may not represent the true probability of the event.

Therefore, model evaluation should distinguish between:

- **Discrimination:** the ability of the model to correctly separate different classes;
- **Calibration:** the agreement between predicted probabilities and observed event frequencies.

A model can achieve strong classification performance while still producing poorly calibrated probabilities.

Calibration can be examined using a calibration plot, where predicted probabilities are compared with observed frequencies. A perfectly calibrated model follows:

$$\text{Observed frequency} = \text{Predicted probability}.$$

When the relationship deviates from this line, the model probabilities can be adjusted using a calibration model fitted between the original predictions and the observed response.

8.1 Calibration example in R

The following example illustrates calibration of a boosted classification model. First, predicted probabilities are obtained from the original model. A calibration model is then fitted using the observed binary response as a function of the predicted probabilities.

The calibration model learns how the original probabilities relate to the observed outcomes and produces corrected probability estimates.

```
Testing = test_data

# gradient boosting
fitted.boost.link <- predict(boost_model, newdata=Testing)
fitted.boost.response <- plogis(fitted.boost.link)

pdata <- data.frame(fits = fitted.boost.response,
                    Outcome = as.numeric(as.character(Testing$Outcome)))

a <- ggplot(pdata, aes(fits, Outcome)) +
  scale_x_continuous("Boosted Classification Trees: fitted values") +
  scale_y_continuous("observed occurrences", limits=c(0, 1), oob = scales::squish) +
  geom_jitter(width=0, height=0.1) + geom_abline(slope=1, intercept=0, col="black") +
  geom_smooth(col="grey60") + theme_bw()

library(scam)
```



```

cali.boost <- scam(Testing$Outcome ~ s(fitted.boost.response, bs="mpi"), family=binomial)

corr.fitted.boost.predict <- cali.boost[["linear.predictors"]]
corr.fitted.boost.response <- plogis(corr.fitted.boost.predict)

pdata.corr <- data.frame(
  corr = corr.fitted.boost.response,
  Outcome = as.numeric(as.character(Testing$Outcome)))

b <- ggplot(pdata.corr, aes(corr, Outcome)) +
  scale_x_continuous("Boosted Classification Trees: corrected values") +
  scale_y_continuous("observed occurrences", limits=c(0, 1), oob = scales::squish) +
  geom_jitter(width=0, height=0.1) + geom_abline(slope=1, intercept=0, col="black") +
  geom_smooth(col="grey60") + theme_bw()

# RandomForest
fitted.rf.prob <- predict(rf_model, newdata=Testing, type = "prob")[,2]

pdata2 <- data.frame(fits = fitted.rf.prob,
  Outcome = as.numeric(as.character(Testing$Outcome)))

c <- ggplot(pdata2, aes(fits, Outcome)) +
  scale_x_continuous("RF Classification Trees: fitted values") +
  scale_y_continuous("observed occurrences", limits=c(0, 1), oob = scales::squish) +
  geom_jitter(width=0, height=0.1) + geom_abline(slope=1, intercept=0, col="black") +
  geom_smooth(col="grey60") + theme_bw()

library(scam)
cali.rf <- scam(Testing$Outcome ~ s(fitted.rf.prob, bs="mpi"), family=binomial)

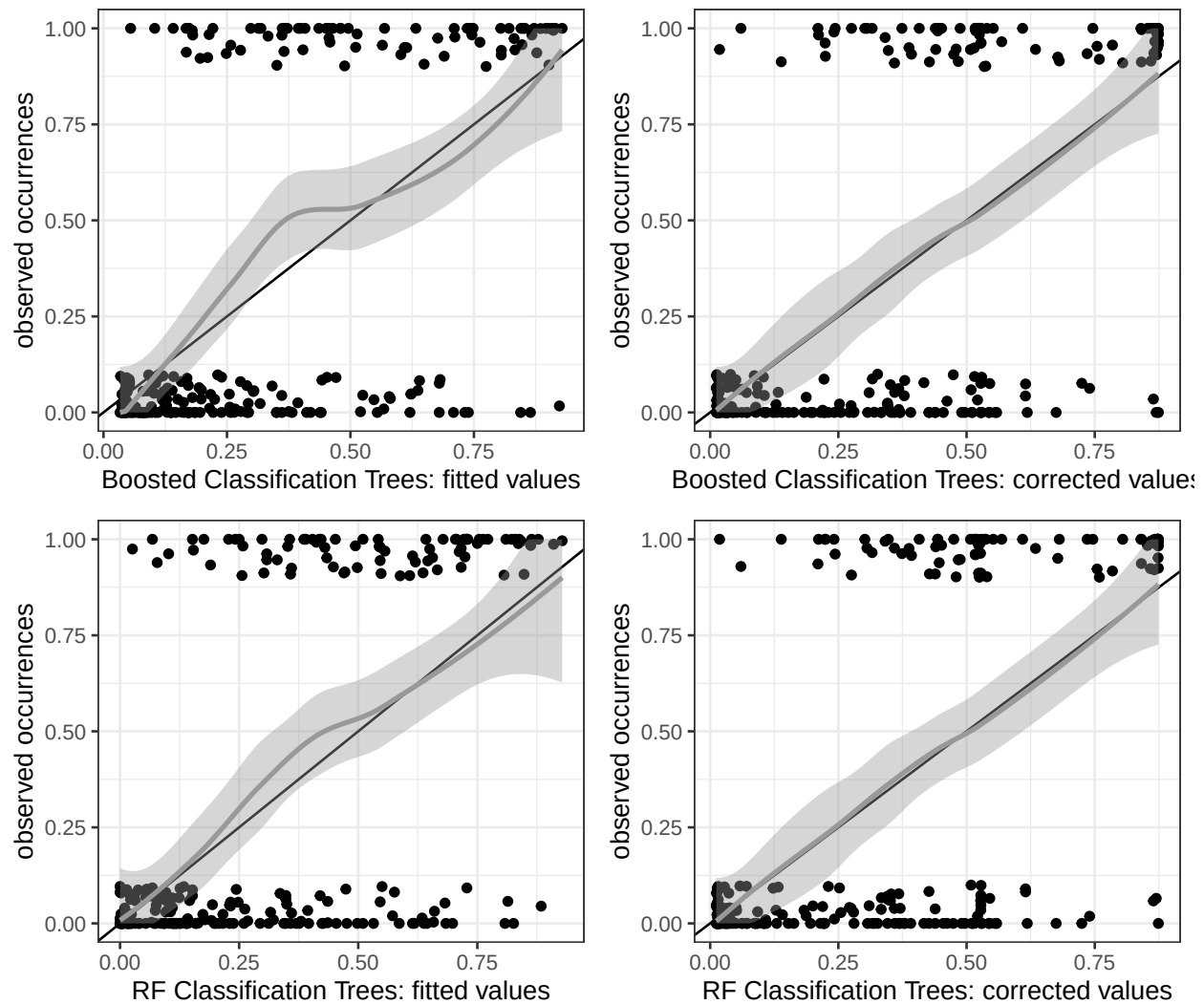
corr.fitted.rf.predict <- cali.rf[["linear.predictors"]]
corr.fitted.rf.response <- plogis(corr.fitted.boost.predict)

pdata.corr2 <- data.frame(corr = corr.fitted.rf.response,
  Outcome = as.numeric(as.character(Testing$Outcome)))

d <- ggplot(pdata.corr2, aes(corr, Outcome)) +
  scale_x_continuous("RF Classification Trees: corrected values") +
  scale_y_continuous("observed occurrences", limits=c(0, 1), oob = scales::squish) +
  geom_jitter(width=0, height=0.1) + geom_abline(slope=1, intercept=0, col="black") +
  geom_smooth(col="grey60") + theme_bw()

library(gridExtra)
grid.arrange(a, b, c, d, ncol = 2)

```



The left panel shows the relationship between the original CART model probabilities and observed outcomes. Deviations from the diagonal line indicate imperfect calibration.

The right panel shows the corrected probabilities after applying the calibration model. Because the calibration model is fitted using the same observations, this result should be interpreted as an illustration of the calibration procedure rather than an independent assessment of model performance.

In practice, after calibration, another dataset (for instance validation data) will be used to assess model performance.

9. Summary and Interpretation of Tree-Based Models

Throughout this lecture, we have seen how tree-based methods evolve from a single decision tree to powerful ensemble learning algorithms.

A single CART model provides an interpretable representation of the relationship between predictors and the response variable. However, individual trees are often unstable because small changes in the training data can produce different tree structures.

Modern ensemble methods improve predictive performance by combining many trees. Bagging and Random Forests reduce prediction variance by averaging independent trees, whereas boosting constructs trees

sequentially so that each new learner improves the current model.

The main characteristics of the methods introduced in this lecture are summarized below.

9.1 Comparison of tree-based methods

Table 4: Summary of tree-based machine-learning methods.

Method	Main principle	Strength
CART	Single recursive tree	Highly interpretable
Bagging	Bootstrap averaging of CARTs	Reduced variance
Weighted CART	Observation-specific weights	Emphasizes important observations
Random Forest	Bagging with random predictor selection	Stable, accurate prediction
Boosting	Sequential improvement of weak learners	High predictive performance
XGBoost	Regularized gradient boosting	Excellent performance for tabular data

9.2 Choosing an appropriate method

The choice of tree-based method depends on the objective of the analysis.

- CART is appropriate when model interpretation is the primary objective.
- Bagging improves prediction by reducing the variability of a single tree.
- Random Forest generally provides a strong balance between predictive accuracy and robustness.
- Weighted CART is useful when some observations or classes should have greater influence during model fitting.
- Boosting and XGBoost often achieve the highest predictive performance but require careful selection of tuning parameters.

No single method is universally best. The most appropriate choice depends on the prediction problem, the available data, and whether interpretation or predictive performance is the primary goal.

9.3 Interpretation of ensemble models

Single CART models are naturally interpretable because the prediction process can be followed through the tree structure and decision rules.

In contrast, ensemble methods such as Random Forests and XGBoost combine many trees, making their predictions much more difficult to interpret directly.

Several model-agnostic techniques have therefore been developed to help explain ensemble predictions, including:

- Variable importance;
- Partial dependence plots (PDPs);
- Individual Conditional Expectation (ICE) plots;
- SHAP values;
- Accumulated Local Effects (ALE) plots.

These interpretation methods provide valuable insight into model behaviour and will be discussed in a later lecture.